

Reviewer Pack — Fourier Coefficient Sum Inverse Proportional to Disjointness...

Entry 982d20a21495 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 01:40:24 UTC

Fourier Coefficient Sum Inverse Proportional to Disjointness Discrepancy

Entry ID: 982d20a21495 Verdict: INCONCLUSIVE Recorded: 2026-05-11 01:40:24 UTC

1. Conjecture

Field A (mathematical branch): Fourier Analysis of Boolean Functions **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For every n -variable DISJOINTNESS instance, the sum of absolute Fourier coefficients of its characteristic function satisfies $\sum |\hat{f}(S)| = \Theta(1/\Delta)$, where Δ is the discrepancy of the communication matrix. Equality holds when the instance is a balanced complete bipartite graph.

Rationale (proposer's reasoning):

Fourier coefficients capture correlation with parity functions, while discrepancy measures communication complexity. Their inverse proportionality suggests hidden structural alignment between spectral properties and combinatorial communication lower bounds.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): df3cdadae5d90b0a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def fast_walsh_hadamard_transform(f):
    n = len(f)
    if n == 1:
        return f
    even = fast_walsh_hadamard_transform(f[0::2])
    odd = fast_walsh_hadamard_transform(f[1::2])
    result = [0] * n
    for k in range(n // 2):
        result[k] = even[k] + odd[k]
        result[k + n // 2] = even[k] - odd[k]
    return result

def discrepancy(G, n):
    def sample_discrepancy():
        s = random.sample(range(1, n), n // 2)
        A = [0] * n
        for i in range(n):
            if i in s:
                A[i] = 1
            else:
                A[i] = -1
        return abs(sum(G[i][j] * (A[i] + A[j]) for i, j in combinations(range(n), 2)))
    return sum(sample_discrepancy() for _ in range(10)) / 10

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    f_hat = fast_walsh_hadamard_transform([sum(G[i][j] for j in range(n)) - n / 2 for i in range(n)])
    metric_value = sum(abs(x) for x in f_hat)
    delta = discrepancy(G, n)
    conjecture_holds = abs(metric_value * delta - 1) < 1e-6
    counterexample = "" if conjecture_holds else "mapping_undefined"
    return {
        "metric_name": "Fourier Coefficient Sum",
```

```

        "metric_value": metric_value,
        "instances_tested": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2**i - 1 for i in range(5, 8)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean = sum(r['metric_value'] for r in results) / len(results)
    std = math.sqrt(sum((r['metric_value'] - mean)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ed": 463, "metric_name": "Fourier Coefficient Sum", "metric_value": 466.0, "instances_tested": 40, "conjecture_holds": true
TRIAL: {"seed": 503, "metric_name": "Fourier Coefficient Sum", "metric_value": 548.0, "instances_tested": 40, "conjecture_holds": true}
TRIAL: {"seed": 547, "metric_name": "Fourier Coefficient Sum", "metric_value": 438.0, "instances_tested": 40, "conjecture_holds": true}
TRIAL: {"seed": 593, "metric_name": "Fourier Coefficient Sum", "metric_value": 476.0, "instances_tested": 40, "conjecture_holds": true}
TRIAL: {"seed": 631, "metric_name": "Fourier Coefficient Sum", "metric_value": 454.0, "instances_tested": 40, "conjecture_holds": true}
TRIAL: {"seed": 677, "metric_name": "Fourier Coefficient Sum", "metric_value": 504.0, "instances_tested": 40, "conjecture_holds": true}
TRIAL: {"seed": 727, "metric_name": "Fourier Coefficient Sum", "metric_value": 564.0, "instances_tested": 40, "conjecture_holds": true}
TRIAL: {"seed": 773, "metric_name": "Fourier Coefficient Sum", "metric_value": 440.0, "instances_tested": 40, "conjecture_holds": true}
TRIAL: {"seed": 821, "metric_name": "Fourier Coefficient Sum", "metric_value": 632.0, "instances_tested": 40, "conjecture_holds": true}
TRIAL: {"seed": 877, "metric_name": "Fourier Coefficient Sum", "metric_value": 504.0, "instances_tested": 40, "conjecture_holds": true}
TRIAL: {"seed": 929, "metric_name": "Fourier Coefficient Sum", "metric_value": 616.0, "instances_tested": 40, "conjecture_holds": true}
RESULT: FALSIFIED counterexample="mapping_undefined" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: Test results show conjecture_holds is False for multiple seeds with counterexample 'mapping_undefined'. The presence of any

counterexample meets the f | next: Investigate the ‘mapping_undefined’ counterexample to determine specific DISJOINTNESS instances violating the conjecture

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111043
2	propose	ollama_remote	qwen3:8b	0	0	103761
3	preregistration	ollama_remote	qwen3:8b	0	0	24207
4	novelty	ollama_remote	qwen3:8b	0	0	20820
5	novelty	ollama_remote	qwen3:8b	0	0	15539
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13081
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9129
8	critic	ollama_remote	qwen3:8b	0	0	29835
9	judge	ollama_remote	qwen3:8b	0	0	15834

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 343249 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/982d20a21495.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/982d20a21495.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/982d20a21495.tar.gz* (if generated)